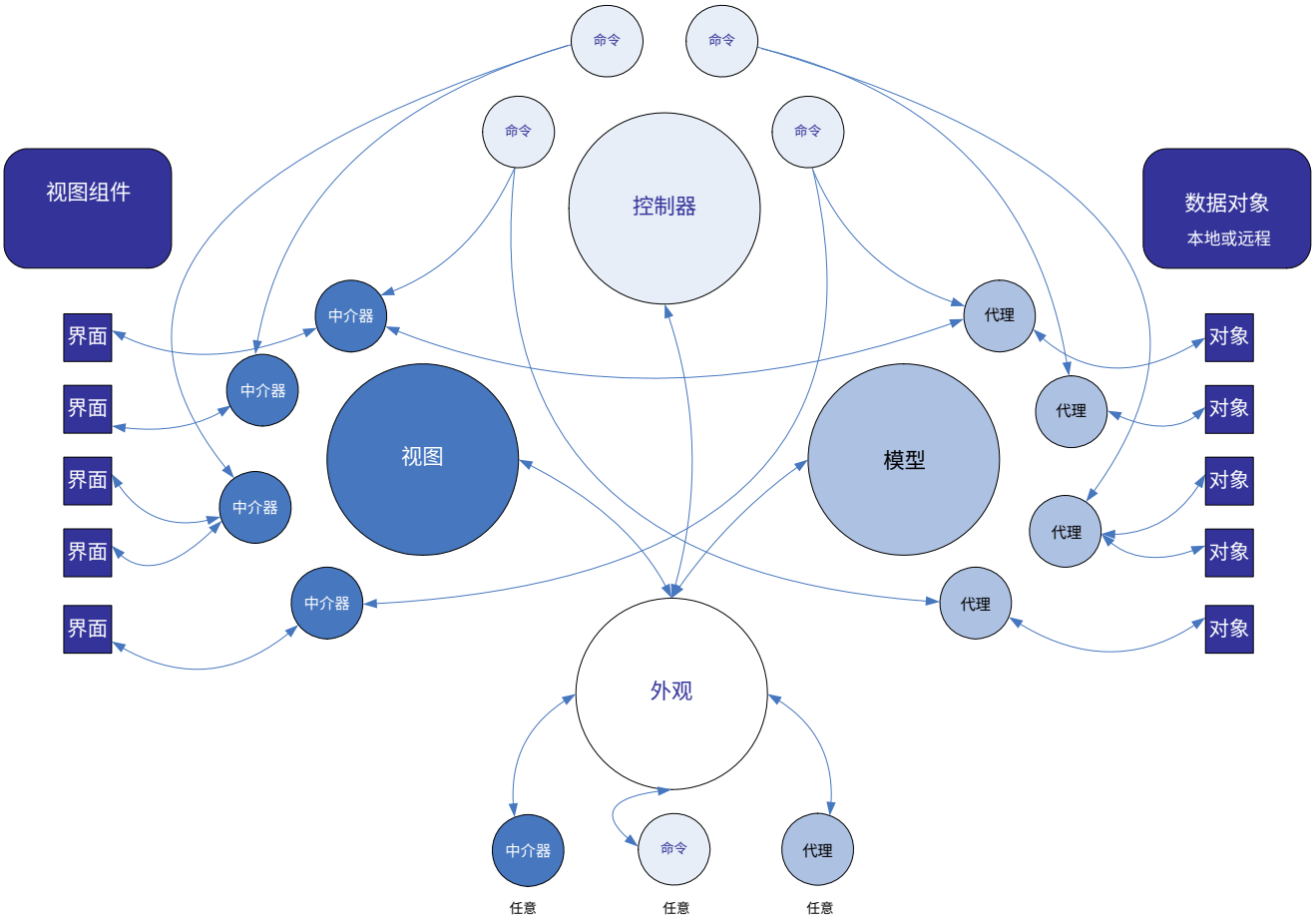
实现
惯用法
与
最佳实践

使用 PureMVC 构建健壮、可扩展、可维护的客户端应用程序
附 ActionScript 3 与 MXML 示例



作者：Cliff Hall <cliff@puremvc.org>
最后修改：2008 年 3 月 2 日

PureMVC 整体结构 4

- Model 与 Proxy 4
- View 与 Mediator 4
- Controller 与 Command 5
- Facade 与核心角色 5
- Observer 与 Notification 5
- 使用通知触发命令执行 6
- 中介器发送通知、声明通知兴趣并接收通知 6
- 代理发送通知，但不接收通知 6

Facade（外观） 7

- 什么是具体的 Facade? 8
- 为应用程序创建具体的 Facade 8
- 初始化具体的 Facade 11

Notification（通知） 13

- 事件与通知 13
- 定义事件和通知常量 14

Command（命令） 16

- 使用宏命令与简单命令 16
- 让命令与中介器、代理保持松耦合 17
- 编排复杂操作与业务逻辑 17

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

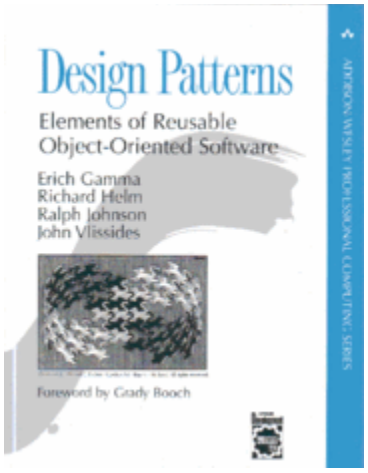
Mediator（中介器） 23

- 具体中介器的职责 23
- 隐式转换视图组件的类型 24
- 监听并响应视图组件 25
- 在具体中介器中处理通知 27
- 中介器与代理及其他中介器的耦合 30
- 用户与视图组件及中介器的交互 32

Proxy（代理） 36

- 具体代理的职责 36
- 隐式转换数据对象的类型 37
- 避免与中介器耦合 39
- 封装领域逻辑 40
- 与远程代理交互 41

灵感来源



PureMVC 是一个基于设计模式的框架，最初源于当时设计高性能 RIA 客户端的现实需求。如今，它已被移植到其他语言和平台，包括服务器环境。本文侧重于客户端。

尽管 PureMVC 在不同受支持平台上的诠释与实现各有特点，它使用的模式在著名的“四人帮”著作《设计模式：可复用面向对象软件的基础》（ISBN 0-201-63361-2）中都有明确的定义。

强烈推荐阅读。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

PureMVC 整体结构

PureMVC 框架的目标非常明确：帮助你将应用程序中不同的编码关注点分离为三个独立的层，即 Model（模型）、View（视图）和 Controller（控制器）。

这种关注点分离，以及让各层协同工作的耦合关系的紧密程度与方向，对构建可扩展、可维护的应用程序至关重要。

在这一经典 MVC 设计元模式的实现中，应用程序的这三层分别由三个单例管理，名称就是 Model、View 和 Controller。单例是只能创建一个实例的类。这三个类统称为“核心角色”。

第四个单例 Facade（外观）为与核心角色通信提供统一接口，从而简化开发。

Model 与 Proxy

Model 只负责缓存以名称标识的 Proxy 引用。Proxy 中的代码操作数据模型，并在需要时与远程服务通信，以持久化或获取数据。

这样就能得到可移植的模型层代码。

View 与 Mediator

View 主要缓存以名称标识的 Mediator 引用。Mediator 中的代码管理视图组件：添加事件监听器，代表组件向系统其他部分发送通知并接收通知，以及直接操作组件的状态。

这就把视图的定义与控制视图的逻辑分离开来。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

PureMVC 整体结构

Controller 与 Command

Controller 维护名称到 Command 类的映射。Command 是无状态的，只在需要时才创建。

Command 可以获取 Proxy 并与其交互、发送通知、执行其他命令，通常用于编排复杂操作或应用程序启动、关闭等系统级活动。应用程序的业务逻辑就放在这里。

Facade 与核心角色

Facade 也是一个单例，它初始化核心角色（Model、View 和 Controller），并提供访问这些角色全部公共方法的统一入口。

通过继承 Facade，应用程序无须直接导入并使用核心角色，就能获得它们提供的全部能力。你只需为应用程序实现一次具体的 Facade，而且这很容易完成。

随后，Proxy、Mediator 和 Command 就可以使用应用程序的这个具体 Facade，相互访问并进行通信。

Observer 与 Notification

PureMVC 应用程序可能运行在无法使用 Flash 的 Event 和 EventDispatcher 类的环境中。因此，框架实现了一套基于观察者的通知机制，使 MVC 核心角色与系统其他部分能够以松耦合的方式通信。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

PureMVC 整体结构

Observer 与 Notification

你无须关心 PureMVC 中 Observer/Notification 的实现细节；它们属于框架内部。你将通过一个简单的方法，从 Proxy、Mediator、Command 以及 Facade 本身发送通知，甚至不必自行创建 Notification 实例。

使用通知触发命令执行

在具体的 Facade 中，命令被映射到通知名称。当相应的通知发出时，Controller 会自动执行映射到它的命令。命令通常负责协调视图与模型关注点之间的复杂交互，同时尽可能少地了解各方的细节。

中介器发送通知、声明通知兴趣并接收通知

当 Mediator 注册到 View 时，View 会调用其 listNotifications 方法，询问它对哪些通知感兴趣。Mediator 必须返回一个数组，其中包含它关注的通知名称。

之后，系统中的任何角色发送同名通知时，相关 Mediator 的 handleNotification 方法就会被调用，并接收到该 Notification 的引用。

代理发送通知，但不接收通知

Proxy 可以出于多种原因发送通知。例如，远程服务代理可以通知系统自己已收到结果；数据发生更新的代理也可以发送变更通知。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

PureMVC 整体结构

代理发送通知，但不接收通知

如果让 Proxy 监听通知，就会使它与视图层和控制器层耦合得过于紧密。视图层和控制器层必须监听代理发出的通知，因为它们的职责就是以可视方式呈现代理持有的数据模型，并让用户与其交互。

然而，视图层和控制器层应当能够独立变化，而不影响数据模型层。例如，一个管理应用程序和与它相关的用户应用程序可能共享相同的模型层类。如果只是用例不同，就可以使用不同的视图／控制器组合，在同一个模型上执行这些用例。

外观

MVC 元模式的三个核心角色，在 PureMVC 中由 Model、View 和 Controller 类表示。为了简化应用程序开发过程，PureMVC 使用了外观模式。

Facade 负责将请求转交给 Model、View 和 Controller，因此你的代码不必导入这些类，你也不必分别与它们打交道。

Facade 类会在构造函数中自动实例化 MVC 核心单例。通常，你会在应用程序中继承框架的 Facade，并用它为 Controller 初始化命令映射。之后，Model 和 View 的准备工作由 Controller 执行的命令来编排。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

外观

什么是具体的 Facade?

尽管核心角色都是完整、可用的实现，但 Facade 提供的实现应当被视为抽象的，因为你永远不会直接实例化它。

你应该继承框架的 Facade，并添加或重写它的一些方法，使它适用于你的应用程序。

随后，通过这个具体的 Facade，访问并通知真正完成系统工作的 Command、Mediator 和 Proxy。按照约定，它名为 ApplicationFacade，不过你也可以使用自己喜欢的名称。

通常，应用程序的视图层次结构（显示组件）由平台惯用的过程创建。在 Flex 中，MXML 应用程序会实例化全部子组件；在 Flash 中，影片会创建舞台上的全部对象。

应用程序的视图层次结构构建完成后，PureMVC 机制随之启动，并准备好模型和视图部分以供使用。

具体的 Facade 还会协助完成启动过程，使主应用程序代码无须了解太多关于其将要连接的 PureMVC 机制的细节。应用程序只需将自身的引用传给具体 Facade 单例上的 startup 方法。

为应用程序创建具体的 Facade

具体的 Facade 只需做很少的工作，就能为应用程序提供强大的能力。请看下面的实现：

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

外观

为应用程序创建具体的 Facade

ApplicationFacade.as:

```
package com.me.myapp
{
	import org.puremvc.as3.interfaces.*;
	import org.puremvc.as3..patterns.facade.*;

	import com.me.myapp.view.*;
	import com.me.myapp.model.*;
	import com.me.myapp.controller.*;

	// MyApp 使用的具体 Facade
	public class ApplicationFacade extends Façade implements IFacade
	{
		// 定义通知名称常量
		public static const STARTUP:String		= "startup";
		public static const LOGIN:String		= "login";

		// ApplicationFacade 单例的工厂方法
		public static function getInstance() : ApplicationFacade
		{
			if ( instance == null ) instance = new ApplicationFacade( );
			return instance as ApplicationFacade;
		}

		// 向 Controller 注册命令
		override protected function initializeController( ) : void
		{
			super.initializeController();
			registerCommand( STARTUP,  StartupCommand );
			registerCommand( LOGIN,  LoginCommand );
			registerCommand( LoginProxy.LOGIN_SUCCESS,  GetPrefsCommand );
		}

		// 启动 PureMVC 机制，并传入应用程序的引用
		public function startup( app:MyApp ) : void
		{
			sendNotification( STARTUP, app );
		}
	}
}
```

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

外观

为应用程序创建具体的 Facade

对于前面的代码，有几点需要注意：

- 它继承 PureMVC 的 Facade 类，而 Facade 实现了 IFacade 接口。

- 它没有重写构造函数。如果重写了，那么在进行任何操作之前，都应先调用父类构造函数。

- 它定义了静态 getInstance 方法，用于返回单例实例，并在必要时创建和缓存该实例。实例的引用保存在父类 Facade 的一个受保护属性中，返回之前必须将其转换为子类类型。

- 它为通知名称定义常量。系统中其他所有角色都通过具体的 Facade 相互访问和通信，因此，具体的 Facade 是定义通知参与者共享的名称常量的理想位置。

- 它为 Controller 初始化命令映射，以便在相应通知发出时执行这些命令。

- 它提供一个 startup 方法。在本例中，该方法接收 MyApp 类型的参数，并通过通知将参数传递给 StartupCommand；该命令注册在通知名称 STARTUP 下。

只要满足这些简单的实现要求，具体的 Facade 就可以从抽象父类继承相当丰富的功能。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

外观

初始化具体的 Facade

PureMVC Facade 的构造函数会调用受保护的方法，初始化 Model、View 和 Controller 实例，并缓存它们的引用。

因此，Facade 通过组合实现并公开 Model、View 和 Controller 的功能：它将这些功能汇集起来，使开发者无须直接与框架的核心角色交互。

那么，在实际应用程序中，Facade 应放在哪里，又该怎样融入整体结构呢？请看下面的 Flex 应用程序：

MyApp.mxml:

```
<?xml version="1.0" encoding="utf-8"?>
<mx:Application xmlns:mx="http://www.adobe.com/2006/mxml"
    creationComplete="façade.startup(this)">

    <mx:Script>
    <![CDATA[
            // 获取 ApplicationFacade
            import com.me.myapp.ApplicationFacade;
            private var facade:ApplicationFacade = ApplicationFacade.getInstance();
    ]]>
    </mx:Script>

    <!—在此定义其余显示层次                -->
</mx:Application>
```

就这些，非常简单。构建初始视图层次结构，获取 ApplicationFacade 实例，然后调用它的 startup 方法。

注意：在 AIR 中，我们会使用 applicationComplete；在 Flash 中，则可以在第 1 帧或独立的文档类中实例化 Facade 并调用 startup。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

外观

初始化具体的 Facade

这个示例中值得注意的要点是：

- 我们以通常的声明式 MXML 方式构建界面：从 <mx:Application> 标签开始，在其中放入组件和容器，它们可以是自定义的，也可以是现成的。
- 使用脚本块声明一个私有变量，并用具体 ApplicationFacade 的单例实例初始化它。
- 由于我们调用静态 ApplicationFacade.getInstance 方法来初始化该变量，因此，当应用程序的 creationComplete 事件触发时，Facade 已经创建，Model、View 和 Controller 也随之创建。不过，此时还没有创建任何 Mediator 或 Proxy。
- 在 Application 标签的 creationComplete 处理函数中，我们调用 startup 方法，并传入主应用程序的引用。请注意，普通视图组件无须了解 Facade，也无须与它交互，但顶层 Application 是例外。顶层 Application（或 Flash 影片）负责构建视图层次结构、初始化 Facade，然后启动 PureMVC 机制。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Notification（通知）

PureMVC 实现了观察者模式，使核心角色及其协作者能够以松耦合且不依赖平台的方式通信。ActionScript 语言本身并不提供 Flex 和 Flash 所用的事件模型；这些事件类来自 Flash 包。框架自行管理内部通信，而不依赖 Flash 平台提供的通信机制，因此得以移植到 C#、J2ME 等其他平台。

Notification 并非简单地替代 Event；它的工作方式与事件有本质区别。通知和事件可以协同工作，构建高度可复用的视图组件。如果设计得当，视图组件甚至不需要知道自己连接的是一个 PureMVC 系统。

事件与通知

事件由实现 IEventDispatcher 接口的 Flash 显示对象派发。事件沿显示层次结构向上“冒泡”，使父对象、父对象的父对象等都有机会处理它。这是一种职责链机制：只有处于这条父子链中的对象才有机会接收并响应事件；除非其他对象持有事件派发者的引用，能够直接在其上设置监听器。

Facade 和 Proxy 发送通知；Mediator 监听并发送通知；Command 与通知建立映射，也可以发送通知。这是一种发布／订阅机制，多个观察者可以接收并处理同一个通知。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Notification（通知）

事件与通知

通知可以带有可选的 body（消息体），其值可以是任意 ActionScript 对象。不同于 Flash 事件，通常没有必要创建自定义 Notification 类，因为它本身就能携带数据。当然，你也可以创建自定义 Notification 类，为相关交互提供强类型支持。不过，针对通知而言，编译期检查在实际中的收益有限，而维护多个通知类又有额外成本；权衡之后，这更多是编程风格的选择。

通知还具有可选的 type（类型），接收者可将其用作区分标记。例如，在文档编辑器中，每个打开的文档都可以对应一个 Proxy 实例，以及一个管理文档编辑视图组件的 Mediator。Proxy 和 Mediator 可以共享一个唯一键，由 Proxy 将它作为通知的 type 传出。

所有注册为关注该 Proxy 通知的 Mediator 实例都会收到通知，但它们会利用 type 属性判断自己是否应该处理它。

定义通知和事件常量

我们已经看到，具体的 Facade 很适合定义公共的通知常量。由于它是与系统交互的中心机制，所有通知参与者默认都会与 Facade 协作。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Notification（通知）

定义通知和事件常量

如果另一个应用程序只需要知道这些常量名称，有时会使用独立的 ApplicationConstants 类来代替具体的 Facade，承担定义常量的工作。

无论采用哪一种方式，集中定义通知名称常量，都能确保通知参与者需要引用通知名称时，可以使用编译器能够检查的类型安全方式；直接使用字符串字面量则可能发生拼写错误，却不会报错。

但不要在具体的 Facade 中定义事件名称。事件名称常量应静态定义在生成事件的边界类中，或者定义在被派发的自定义 Event 类中。

视图组件和数据对象代表应用程序的实际边界。如果它们通过派发事件，与相关的 Mediator 或 Proxy 通信，而不是调用方法或发送通知，就能保持可复用性。

如果视图组件或数据对象派发了其管理者 Mediator 或 Proxy 正在监听的事件，那么通常只有这一对协作者需要知道具体的事件名称。监听者与 PureMVC 系统其他部分之间的进一步通信，可以通过通知完成。

虽然这些协作对（Mediator／视图组件，以及 Proxy／数据对象）之间的关系必然比较紧密，但它们与应用程序架构的其余部分保持松耦合。因此，在需要重构数据模型或用户界面时，变动能够限制在更小的范围内。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Command（命令）

具体的 Facade 通常会用启动时所需的一组“通知到命令”的映射初始化 Controller。对于每一个映射，Controller 都会将自身注册为相应通知的观察者。收到通知后，Controller 会实例化对应的 Command。最后，Controller 调用命令的 execute 方法，并将通知传给它。

命令是无状态的：需要时创建，执行完毕后就应退出。因此，不要在生命周期较长的对象中实例化命令或保存命令的引用，这一点很重要。

使用宏命令与简单命令

和所有 PureMVC 框架类一样，命令也实现了一个接口，即 ICommand。PureMVC 提供了两种便于扩展的 ICommand 实现。

SimpleCommand 类只有一个 execute 方法，接收 INotification 实例。把你的代码放进 execute 方法即可。

MacroCommand 类允许你按顺序执行多个子命令。每个子命令都会被创建，并接收原始通知的引用。

MacroCommand 在构造函数中调用 initializeMacroCommand 方法。你可以在子类中重写该方法，为每个需要添加的命令调用一次 addSubCommand。SimpleCommand 和 MacroCommand 可以任意组合添加。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Command（命令）

让命令与中介器、代理保持松耦合

通知发出后，由 Controller 执行对应的命令。除 Controller 之外，其他任何角色都不应实例化并执行命令。为了与系统的其他部分通信和交互，命令可以:

- 注册或移除 Mediator、Proxy 和 Command，或检查它们是否已经注册。
- 发送由其他 Command 或 Mediator 响应的通知。
- 获取 Proxy 和 Mediator，并直接操作它们。

命令让我们能够轻松地使视图中的元素进入适当的状态，或向视图各处传递数据。命令还可以对模型执行涉及多个 Proxy 的事务性交互，在整个事务完成后发送通知；也可以处理异常，并在失败时采取措施。

编排复杂操作与业务逻辑

应用程序中有几个可以放置代码的位置：Command、Mediator 和 Proxy。于是，一个问题必然会反复出现：什么代码应放在哪里？一个 Command 究竟应该做什么?

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Command（命令）

编排复杂操作与业务逻辑

首先，要区分应用程序中的业务逻辑与领域逻辑。

Command 承载应用程序的业务逻辑，即从技术上实现应用程序需要围绕领域模型执行的用例。这包括协调模型和视图的状态。

模型通过 Proxy 维护自身完整性。Proxy 承载领域逻辑，并公开用于操作数据对象的 API。无论数据模型位于客户端还是服务器端，Proxy 都会封装对它的全部访问。因此，对应用程序的其他部分而言，真正需要关注的只是数据能够同步访问，还是必须异步访问。

Command 可以用于编排复杂的系统行为：这些行为必须按特定顺序发生，而且某个操作的结果可能成为下一个操作的输入。

Mediator 和 Proxy 应当向 Command 以及彼此提供粗粒度接口，隐藏其管理的视图组件或数据对象的实现。请注意，这里说的视图组件，是指用户直接交互的按钮或控件；数据对象既包括保存数据的任意结构，也包括用于获取或存储这些数据的远程服务。

Command 与 Mediator、Proxy 交互，但应与边界的具体实现隔离。下面这些命令用于准备系统，使其可以投入使用：

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Command（命令）

编排复杂操作与业务逻辑

StartupCommand.as:

```
package com.me.myapp.controller
{
    import org.puremvc.as3.interfaces.*;
    import org.puremvc.as3.patterns.command.*;

    import com.me.myapp.controller.*;

    // 应用程序启动时执行的 MacroCommand。
    public class StartupCommand extends MacroCommand
    {
        // 通过添加子命令来初始化 MacroCommand。
       override protected function initializeMacroCommand() : void
       {
              addSubCommand( ModelPrepCommand );
              addSubCommand( ViewPrepCommand );
       }
    }
}
```

这个 MacroCommand 添加了两个子命令。宏命令执行时，子命令按先进先出（FIFO）的顺序执行。这样就得到一个顶层的启动操作“队列”。

那么，具体该做些什么，又该按什么顺序做呢?

在向用户呈现应用程序的数据，或允许用户与数据交互之前，必须先使模型处于一致且已知的状态。完成这一步后，就可以准备视图，让它呈现模型的数据，并允许用户操作和交互。

因此，启动过程通常包含两大类活动：先准备模型，再准备视图。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Command（命令）

编排复杂操作与业务逻辑

ModelPrepCommand.as:

```
package com.me.myapp.controller
{
    import org.puremvc.as3.interfaces.*;
    import org.puremvc.as3.patterns.observer.*;
    import org.puremvc.as3.patterns.command.*;

    import com.me.myapp.*;
    import com.me.myapp.model.*;

    // 创建 Proxy，并向 Model 注册。
    public class ModelPrepCommand extends SimpleCommand
    {
        // 由 MacroCommand 调用
        override public function execute( note : INotification ) : void
        {
                facade.registerProxy( new SearchProxy() );
                facade.registerProxy( new PrefsProxy() );
                facade.registerProxy( new UsersProxy() );
        }
    }
}
```

准备模型通常很简单：创建并注册系统启动时所需的全部 Proxy。

上面的 ModelPrepCommand 是一个 SimpleCommand，用于准备模型以供使用。它是前面那个 MacroCommand 的第一个子命令，因此最先执行。它通过具体的 Facade 创建并注册系统启动时所需的各种 Proxy 类。

请注意，这个命令不会操作或初始化模型数据。为使数据对象可以供系统使用，任何必要的数据获取、创建或初始化都应由 Proxy 负责。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Command（命令）

编排复杂操作与业务逻辑

ViewPrepCommand.as:

```
package com.me.myapp.controller
{
    import org.puremvc.as3.interfaces.*;
    import org.puremvc.as3.patterns.observer.*;
    import org.puremvc.as3.patterns.command.*;

    import com.me.myapp.*;
    import com.me.myapp.view.*;

    // 创建 Mediator，并向 View 注册。
    public class ViewPrepCommand extends SimpleCommand
    {
        override public function execute( note : INotification ) : void
        {
            var app:MyApp = note.getBody() as MyApp;
            facade.registerMediator( new ApplicationMediator( app ) );
        }
    }
}
```

这是一个准备视图以供使用的 SimpleCommand。它是前面那个 MacroCommand 的最后一个子命令，因此最后执行。注意，它只创建并注册了 ApplicationMediator，该中介器管理 Application 视图组件。

此外，它还把通知的 body 传给 Mediator 的构造函数。这个 body 是 Application 的引用，由应用程序自身在最初发送 STARTUP 通知时作为消息体传出。（参见前面的 MyApp 示例。）

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Command（命令）

编排复杂操作与业务逻辑

Application 是一个比较特殊的视图组件：它会实例化启动时初始化的其他全部视图组件，并将它们作为子组件持有。

视图组件需要 Mediator，才能与系统的其他部分通信。而创建这些 Mediator，又需要获得它们所管理的视图组件的引用；在这一时刻，只有 Application 知道这些引用。

我们只允许 Application 的 Mediator 了解 Application 的具体实现，因此，剩余 Mediator 的创建工作放在 Application 的 Mediator 构造函数中处理。

这样，通过上述三个命令，我们就编排好了模型与视图的有序初始化。完成这些工作时，命令无须了解太多模型或视图的细节。

当模型细节或视图实现发生变化时，可以按需重构 Proxy 和 Mediator。命令中的业务逻辑应当与应用程序边界处的重构隔离。

模型应封装“领域逻辑”，在 Proxy 中维护数据完整性。Command 针对模型执行“事务逻辑”或“业务逻辑”，封装对跨多个 Proxy 的事务的协调，或者按应用程序所要求的方式处理并报告异常。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Mediator（中介器）

Mediator 类负责协调用户与应用程序中一个或多个视图组件（例如 Flex 的 DataGrid 或 Flash 的 MovieClip）之间的交互，并将这些组件与 PureMVC 应用程序的其他部分联系起来。

在基于 Flash 的应用程序中，Mediator 通常会在其视图组件上设置事件监听器，处理用户操作以及组件发出的数据请求。它通过发送和接收通知，与应用程序的其他部分通信。

具体中介器的职责

Flash、Flex 和 AIR 框架提供了大量具有丰富交互能力的 UI 组件。你可以扩展这些组件，也可以用 ActionScript 自行编写组件，从而以无数种方式向用户呈现数据模型，并允许用户与其交互。

在不久的将来，还会有其他能够运行 ActionScript 的平台。这个框架也已经移植到 Silverlight、J2ME 等其他平台，并有相应演示，进一步拓宽了使用这项技术开发 RIA 的空间。

PureMVC 的一个目标，是对应用程序边界所使用的技术保持中立，并提供简单的惯用法，以适配你当前需要处理的任何 UI 组件或数据结构／服务。

对于基于 PureMVC 的应用程序而言，任何 UI 组件都可以成为视图组件，无论它由哪个框架提供，也无论内部包含多少子组件。视图组件应尽可能封装自身的状态和操作，只公开由事件、方法和属性组成的简单 API。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Mediator（中介器）

具体中介器的职责

具体的 Mediator 通过独占一个或多个视图组件的引用，并与这些组件公开的 API 交互，帮助我们将组件适配到应用程序中。

Mediator 的主要职责是处理视图组件派发的事件，以及系统其他部分发出的相关通知。

由于 Mediator 也经常与 Proxy 交互，因此，通常会在 Mediator 的构造函数中获取经常访问的 Proxy，并在本地保存其引用。这样可以避免为了取得同一个引用而反复调用 retrieveProxy。

隐式转换视图组件的类型

PureMVC 提供的 Mediator 基类，其构造函数只接收一个名称和一个通用的 Object。具体 Mediator 的构造函数会把视图组件传给父类；随后，组件立即可在类内部通过名为 viewComponent 的受保护属性访问，该属性的通用类型为 Object。

Mediator 构造完成后，你也可以调用 setViewComponent 方法，动态设置它的视图组件。

无论以哪种方式设置，访问组件公开的 API 时，你经常需要将这个 Object 转换为组件的实际类型。这种操作既繁琐又重复。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Mediator（中介器）

隐式转换视图组件的类型

ActionScript 提供一种称为隐式 getter 和 setter 的语言特性。隐式 getter 看起来像方法，但对类或应用程序的其他部分来说，它以属性的形式公开。这一特性非常适合解决反复进行类型转换的问题。

在具体的 Mediator 中，一个实用的惯用法是：定义一个隐式 getter，将视图组件转换为实际类型，并为它起一个有意义的名称。可以创建如下方法：

```
protected function get controlBar() : MyAppControlBar
{
        return viewComponent as MyAppControlBar;
}
```

然后，在 Mediator 的其他位置，就不必再这样写：

```
MyAppControlBar ( viewComponent ).searchSelction =
MyAppControlBar.NONE_SELECTED;
```

而是可以改为：

```
controlBar.searchSelction = MyAppControlBar.NONE_SELECTED;
```

监听并响应视图组件

一个 Mediator 通常只管理一个视图组件，但也可能管理多个，例如 ApplicationToolBar 及其包含的按钮或控件。

我们可以将一组相关控件（例如一个表单）放在同一个视图组件中，并以属性的形式向 Mediator 公开子组件。不过，最好尽可能封装组件的实现。让组件通过一个自定义类型的对象交换数据，会是更好的方式。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Mediator（中介器）

监听并响应视图组件

Mediator 负责与控制器层和模型层交互，并在收到相关通知时更新视图组件。

在 Flash 平台上，通常会在 Mediator 构造时，或调用其 setViewComponent 方法时，在视图组件上设置事件监听器，并指定一个处理方法：

```
controlBar.addEventListener( AppControlBar.BEGIN_SEARCH, onBeginSearch );
```

Mediator 如何响应这个事件，当然完全取决于当前需求。通常，具体 Mediator 的事件处理方法会组合执行以下操作：

- 检查事件的类型，或检查预期包含的自定义内容。
- 检查或修改视图组件公开的属性，或调用其公开的方法。
- 检查或修改 Proxy 公开的属性，或调用其公开的方法。
- 发送一个或多个通知，由其他 Mediator 或 Command 响应，也可能由同一个 Mediator 实例响应。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Mediator（中介器）

监听并响应视图组件

以下几条经验规则很有帮助：

- 如果多个其他 Mediator 都需要参与对某个事件的整体响应，就更新一个公共 Proxy，或发送一条通知，让感兴趣的 Mediator 各自作出适当响应。

- 如果需要与其他 Mediator 进行大量协调交互，较好的做法是使用一个 Command，把各个步骤集中写在一处。

- 获取其他 Mediator 并直接操作它们，或者让 Mediator 公开这类操作方法，都应视为不良做法。

- 若要操作应用程序状态信息并将其分发给 Mediator，应在专门维护状态的 Proxy 上设置属性或调用方法，再让 Mediator 关注这些状态 Proxy 发出的通知。

在具体中介器中处理通知

给视图组件添加事件监听器需要显式操作，而将 Mediator 接入 PureMVC 系统的过程既简单又自动。

Mediator 注册到 View 时，会被询问对哪些通知感兴趣。它返回一个数组，其中包含它希望处理的通知名称。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Mediator（中介器）

在具体中介器中处理通知

最简单的回应方式，是用一个表达式创建并返回匿名数组，把通知名称放入其中。通知名称应定义为静态常量，通常放在具体的 Facade 中。

定义 Mediator 关注的通知列表十分简单：

```
override public function listNotificationInterests() : Array
{
    return [
              ApplicationFacade.SEARCH_FAILED,
              ApplicationFacade.SEARCH_SUCCESS
            ];
}
```

系统中的任何角色，包括 Mediator 自己，只要发送了这个 Mediator 声明关注的通知，其 handleNotification 方法就会被调用，并接收到该通知。

考虑到可读性，以及重构时增加或移除通知处理的便利性，在 handleNotifications 方法中，优先推荐 switch / case 结构，而不是 if / else if 的表达式风格。

原则上，响应任意一条通知时需要完成的工作都应很少，而且所需信息应包含在通知本身中。偶尔可以根据通知中的信息，从 Proxy 获取一些数据，但通知处理函数中不应该出现复杂逻辑。如果出现了，就说明你正试图把本应属于 Command 的业务逻辑，放进 Mediator 的通知处理函数中。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Mediator（中介器）

在具体中介器中处理通知

```
override public function handleNotification( note : INotification ) : void
{
    switch ( note.getName() )
    {
            case ApplicationFacade.SEARCH_FAILED:
                    controlBar.status = AppControlBar.STATUS_FAILED;
                    controlBar.searchText.setFocus();
                    break;

            case ApplicationFacade.SEARCH_SUCCESS:
                    controlBar.status = AppControlBar.STATUS_SUCCESS;
                    break;

            }
    }
}
```

此外，典型 Mediator 的 handleNotification 方法处理的通知通常不超过 4 或 5 种。超过这个数量，就说明它的职责应该进一步细分。应为视图组件的子组件创建 Mediator，而不是试图用一个庞大的 Mediator 包办所有事情。

使用一个预先确定的通知处理方法，是 Mediator 监听事件与监听通知之间的关键区别。

对于事件，我们会有多个处理方法，通常每种事件对应一个。这些方法一般只发送通知。它们不应复杂，也不应过度管理视图组件的众多细节，因为视图组件本身应封装实现细节，向 Mediator 公开粗粒度 API。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Mediator（中介器）

在具体中介器中处理通知

对于通知，只有一个处理方法，Mediator 关注的全部通知都在这个方法中处理。

最好将所有通知响应完整地放在 handleNotification 方法内部，使用 switch 语句按通知名称清晰地区分各个分支。

关于 switch/case 的使用，一直有不少争论。许多开发者认为所有 case 都在同一个方法作用域中执行，限制过大。不过，采用单一通知处理方法和 switch/case 风格，正是为了限制 Mediator 内部所做的事情；它仍然是推荐的结构。

Mediator 的职责，是协调视图组件与系统其他部分之间的通信。

不妨想象一位翻译，在联合国会议上帮助她所服务的大使与其他成员交流。她通常只需要做简单的翻译和消息转达，偶尔查找一个恰当的比喻或事实。PureMVC 中 Mediator 的职责也是如此。

中介器与代理及其他中介器的耦合

视图最终需要以图形化、可交互的方式呈现数据模型，因此，它与应用程序中的 Proxy 存在较紧密的单向耦合是正常的。视图必须了解模型，但模型不需要知道视图的任何细节。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Mediator（中介器）

中介器与代理及其他中介器的耦合

Mediator 可以自由地从 Model 获取 Proxy，并通过 Proxy 公开的 API 读取或操作数据对象。不过，如果把这些工作放在 Command 中，视图与模型之间的耦合就会更松。

同样，Mediator 也可以从 View 获取其他 Mediator 的引用，通过对方公开的接口读取或操作它们。

但这不是一种良好做法，因为它会在视图的不同部分之间建立依赖，导致重构一个部分时难以避免影响另一个部分。

如果一个 Mediator 想与视图的其他部分通信，应发送通知，而不是获取另一个 Mediator 并直接操作它。Mediator 不应公开操作其所管理视图组件的方法，而应只通过响应通知来完成这类工作。

如果一个 Mediator 在响应事件或通知时，大量操作视图组件的内部细节，应将这些工作重构为视图组件自身的方法，尽可能封装其实现，以提高可复用性。

如果一个 Mediator 大量操作 Proxy 或其中的数据，应将这些工作重构到 Command 中。这样既能简化 Mediator，也能把业务逻辑放在可由其他视图部分复用的 Command 中，并最大限度降低视图与模型的耦合。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Mediator（中介器）

用户与视图组件及中介器的交互

考虑一个用于显示表单的 LoginPanel 组件。与它配套的 LoginPanelMediator 允许用户通过 LoginPanel 提交凭据和登录意图，并通过发起登录尝试来响应用户输入。

LoginPanel 与 LoginPanelMediator 的协作方式是：用户输入凭据并希望尝试登录时，组件发出 TRY_LOGIN 事件。LoginPanelMediator 处理这个事件时，发送一条通知，并将组件中已经填写好的 LoginVO 作为消息体。

LoginPanel.mxml:

```
<?xml version="1.0" encoding="utf-8"?>
<mx:Panel xmlns:mx="http://www.adobe.com/2006/mxml"
    title="Login" status="”{loginStatus}”>

    <!—
    此组件派发的事件。遗憾的是，这里不能使用常量名称，
    因为 MetaData（元数据）是编译器指令。
     -->
    <mx:MetaData>
            [Event('tryLogin')];
    </mx:MetaData>

    <mx:Script>
    <![CDATA[
            import com.me.myapp.model.vo.LoginVO;
            // 表单字段与这个对象的属性进行双向绑定
            [Bindable] public var loginVO:LoginVO = new LoginVO();
            [Bindable] public var loginStatus:String = NOT_LOGGED_IN;

            // 在视图组件中定义事件名称常量
            public static const TRY_LOGIN:String='tryLogin';
            public static const LOGGED_IN:String='Logged In';
            public static const NOT_LOGGED_IN:String='Enter Credentials';
    ]]>
    </mx:Script>
```

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Mediator（中介器）

用户与视图组件及中介器的交互

```
    <mx:Binding source="username.text" destination="loginVO.username"/>
    <mx:Binding source="password.text" destination="loginVO.password"/>

    <!—登录表单       -->
    <mx:Form id="loginForm" >
        <mx:FormItem label="Username:">
                <mx:TextInput id="username" text="{loginVO.username}" />
        </mx:FormItem>
        <mx:FormItem label="Password:">
                <mx:TextInput id="password" text="{loginVO.password}"
                displayAsPassword="true" />
        </mx:FormItem>
        <mx:FormItem >
            <mx:Button label="Login" enabled="{loginStatus == NOT_LOGGED_IN}"
                click="dispatchEvent( new Event(TRY_LOGIN, true ));"/>
        </mx:FormItem>
  </mx:Form>
</mx:Panel>
```

LoginPanel 视图组件根据用户的表单输入填充一个新的 LoginVO。当用户点击 Login 按钮时，组件派发一个事件，随后由 LoginPanelMediator 接手处理。

这样，视图组件只需负责收集数据，并在数据准备好时通知系统。更完善的组件还应仅在用户名和密码字段都非空时启用登录按钮，从而在无效登录尝试发起之前就将其阻止。

视图组件隐藏了内部实现。Mediator 所使用的完整 API，仅包括 TRY_LOGIN 事件、可供检查的 LoginVO 属性，以及 Panel 的 status 属性。

LoginPanelMediator 还会响应 LOGIN_FAILED 和 LOGIN_SUCCESS 通知，并设置 LoginPanel 的状态。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Mediator（中介器）

用户与视图组件及中介器的交互

LoginPanelMediator.as:

```
package com.me.myapp.view
{
	import flash.events.Event;
	import org.puremvc.as3.interfaces.*;
	import org.puremvc.as3.patterns.mediator.Mediator;
	import com.me.myapp.model.LoginProxy;
	import com.me.myapp.model.vo.LoginVO;
	import com.me.myapp.ApplicationFacade;
	import com.me.myapp.view.components.LoginPanel;

	// 用于与 LoginPanel 组件交互的 Mediator。
	public class LoginPanelMediator extends Mediator implements IMediator
	{

		public static const NAME:String = 'LoginPanelMediator';

		public function LoginPanelMediator( viewComponent:LoginPanel )
		{
			super( NAME, viewComponent );
			LoginPanel.addEventListener( LoginPanel.TRY_LOGIN, onTryLogin );
		}

		// 列出关注的通知
		override public function listNotificationInterests( ) : Array
		{
			return [
					LoginProxy.LOGIN_FAILED,
					LoginProxy.LOGIN_SUCCESS
				];
		}

		// 处理通知
		override public function handleNotification( note:INotification ):void
		{
			switch ( note.getName() )  {
				case LoginProxy.LOGIN_FAILED:
					LoginPanel.loginVO = new LoginVO( );
					loginPanel.loginStatus = LoginPanel.NOT_LOGGED_IN;
					break;
```

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Mediator（中介器）

用户与视图组件及中介器的交互

```
                        case LoginProxy.LOGIN_SUCCESS:
                                loginPanel.loginStatus = LoginPanel.LOGGED_IN;
                                break;
                }
        }

        // 用户点击了登录按钮；尝试登录
        private function onTryLogin ( event:Event ) : void  {
                sendNotification( ApplicationFacade.LOGIN, loginPanel.loginVO );
        }

        // 将 viewComponent 转换为它的实际类型。
        protected function get loginPanel() : LoginPanel  {
                return viewComponent as LoginPanel;
        }
    }
}
```

请注意，LoginPanelMediator 在构造函数中为 LoginPanel 添加了事件监听器，因此，当用户点击登录按钮时，onTryLogin 方法会被调用。

在 onTryLogin 方法中，发送 LOGIN 通知，并携带用户填写好的 LoginVO。此前，我们已经将 LoginCommand 注册到这个通知；它会调用 LoginProxy 的 login 方法，并传入 LoginVO。LoginProxy 随后向远程服务尝试登录，并发送 LOGIN_SUCCESS 或 LOGIN_FAILED 通知。这些类的定义位于 Proxy 一节的末尾。

LoginPanelMediator 将 LOGIN_SUCCESS 和 LOGIN_FAILED 列为关注的通知，因此无论结果如何，它都会收到通知。成功时，它将 LoginPanel 的 loginStatus 设为 LOGGED_IN；失败时，设为 NOT_LOGGED_IN，并清空 LoginVO。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Proxy（代理）

一般而言，代理模式通过为对象提供一个替身，控制对该对象的访问。在基于 PureMVC 的应用程序中，Proxy 类专门用于管理应用程序数据模型的一部分。

Proxy 可以管理对本地创建的数据结构的访问，无论这个结构多么复杂。这个结构就是 Proxy 的数据对象。在这种情况下，与它交互的惯用法通常包括同步地设置和获取数据。Proxy 可以公开其数据对象的全部或部分属性和方法，也可以公开数据对象本身的引用。在公开更新数据的方法时，它还可以向系统其他部分发送通知，告知数据已经改变。

远程代理可用于封装与远程服务的交互，以保存或获取数据。Proxy 可以持有与远程服务通信的对象，并控制对服务收发数据的访问。在这种情况下，调用方可以设置 Proxy 的数据或调用它的方法，然后等待异步通知。当服务收到远程端点返回的数据时，Proxy 就会发送这条通知。

具体代理的职责

具体的 Proxy 通过管理数据对象，以及应用程序对该对象的访问，封装数据模型的一部分，无论数据来自哪里、属于什么类型。

PureMVC 提供的 Proxy 实现类，是一个简单的数据承载对象，可以向 Model 注册。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Proxy（代理）

具体代理的职责

尽管这种形式本身已完全可用，你通常仍会继承 Proxy，并为具体的代理添加专门功能。代理模式的常见变体包括:

- 远程代理（Remote Proxy）：具体代理管理的数据位于远程，需要通过某种服务访问。
- 代理与委托（Proxy and Delegate）：当多个 Proxy 需要共享一个服务对象时使用。Delegate 类维护服务对象并控制对它的访问，确保响应被正确路由给请求者。
- 保护代理（Protection Proxy）：在对象需要具有不同访问权限时使用。
- 虚拟代理（Virtual Proxy）：按需创建开销较大的对象。
- 智能代理（Smart Proxy）：首次访问时将数据对象加载到内存，进行引用计数，并允许锁定对象，以确保其他对象不能修改它。

隐式转换数据对象的类型

PureMVC 提供的 Proxy 基类接受名称和通用 Object 作为构造函数参数。Proxy 构造完成后，你可以调用 setData 方法，动态设置其数据对象。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Proxy（代理）

隐式转换数据对象的类型

与 Mediator 及其视图组件的情况一样，为访问这个 Object 公开的属性和方法，你需要经常把它转换为实际类型。这至少会带来繁琐重复的操作，还可能导致代码采用不恰当的惯用法，暴露过多的数据对象实现细节。

另外，数据对象通常是复杂结构。除了需要一个转换为实际类型的结构引用，我们往往还需要为其中的若干部分提供便于使用的具名引用。

ActionScript 的隐式 getter 和 setter 特性，在解决频繁类型转换和不恰当地了解实现细节的问题上，再次发挥了作用。

在具体的 Proxy 中，一个实用的惯用法是定义隐式 getter，将数据对象转换为实际类型，并为它起一个有意义的名称。此外，还可以定义多个不同类型的 getter，以获取数据对象的特定部分。例如：

```
public function get searchResultAC () : ArrayCollection
{
        return data as ArrayCollection;
}

public function get resultEntry( index:int ) : SearchResultVO
{
        return searchResultAC.getItemAt( index )  as SearchResultVO;
}
```

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Proxy（代理）

隐式转换数据对象的类型

在 Mediator 的其他位置，就不必这样写：

```
var item:SearchResultVO =
ArrayCollection ( searchProxy.getData() ).lastResult.getItemAt( 1 ) as SearchResultVO;
```

而是可以改为：

```
var item:SearchResultVO = searchProxy.resultEntry( 1 );
```

避免与中介器耦合

Proxy 不像 Mediator 那样会被询问关注哪些通知，也从不会被通知，因为它不应关心视图的状态。相反，Proxy 通过公开方法和属性，让其他角色操作它。

具体的 Proxy 不应通过获取并操作 Mediator，来告知系统其数据对象发生了变化。它应该发送通知，由 Command 或 Mediator 响应。通知会如何影响系统，不应成为 Proxy 需要考虑的事情。

让模型层完全不了解系统的具体实现，通常就能在不影响模型层的情况下重构视图层和控制器层。反过来则不完全成立：模型层很难在发生变化时，不影响视图层，以及可能受影响的控制器层。毕竟，这两层存在的目的，就是让用户与模型层交互。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Proxy（代理）

在 Proxy 中封装领域逻辑

模型层的变化几乎总会导致视图层／控制器层进行一些重构。为了进一步分离模型层与视图层、控制器层共同关注的内容，应确保尽可能把领域逻辑放在 Proxy 中。

Proxy 不仅可以控制对数据的访问，还可以按需要操作数据，使其保持有效状态。

例如，计算销售税属于领域逻辑，应该放在 Proxy 中，而不是 Mediator 或 Command 中。尽管这些位置都能执行计算，但放在 Proxy 中不仅合理，还能让其他层更轻量、更易于重构。

Mediator 可以获取 Proxy，调用它的销售税计算函数，并传入一些表单项。但如果将实际计算过程放在 Mediator 中，就是把领域逻辑嵌入了视图层。销售税计算是属于领域模型的规则。视图只需把它看作领域模型提供的一种能力：只要具备适当输入，就可以使用。

假设你正在开发的应用程序，是一个在浏览器中运行、面向桌面分辨率的 RIA。现在需要推出面向 PDA 分辨率的新版本，缩减用例集合，但仍保留现有应用程序全部模型层需求。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Proxy（代理）

在 Proxy 中封装领域逻辑

只要正确地分离关注点，我们就可能完整复用模型层，只需为它适配新的视图层和控制器层。

在实现时，把销售税的实际计算放进 Mediator，或许显得高效或方便：你刚从表单取得数据，可能正想计算销售税，再把结果作为订单写入模型。

然而，这样一来，应用程序的每个版本都需要重复实现，或把销售税逻辑复制粘贴到全新的、完全不同的视图层中。它将无法随着模型层类库的引入而自动获得。

与远程代理交互

远程代理只是从远程位置获取数据对象的 Proxy。这通常意味着我们需要以异步方式与它交互。

Proxy 如何获取数据，取决于客户端平台、远程服务的实现，以及开发者的偏好。在 Flash/Flex 环境中，我们可以在 Proxy 内使用 HTTPService、WebService、RemoteObject、DataService，甚至 XMLSocket 发起服务请求。

根据具体需求，远程代理可以在属性被设置或方法被调用时动态发送请求；也可以只在构造时请求一次，之后提供对这些数据的 get/set 访问。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Proxy（代理）

与远程代理交互

可以在 Proxy 中采用多种优化，提高与远程服务通信的效率。例如，缓存数据以减少频繁的网络交互；或者只发送数据结构中发生变化的部分，从而减少带宽消耗。

当系统中的其他角色动态调用远程代理发起请求时，Proxy 应在结果返回后发送通知。关注这个通知的角色，可能是最初发起请求的角色，也可能不是。

例如，调用远程服务进行搜索并显示结果，可以按以下步骤进行：

- 视图组件通过派发事件发起搜索。
- 它的 Mediator 作出响应：获取适当的远程代理，并设置 searchCriteria 属性。

- Proxy 的 searchCriteria 属性实际上是一个隐式 setter。它保存该值，并通过内部的 HTTPService 发起搜索；Proxy 同时监听这个服务的结果事件和故障事件。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Proxy（代理）

与远程代理交互

- 服务返回时派发 ResultEvent。Proxy 响应这个事件，将结果设置到自身的一个公共属性中。

- 随后，Proxy 发送表示成功的通知，并将数据对象的引用放入通知的 body。

- 另一个 Mediator 已经声明关注这条通知。它作出响应：从通知的 body 获取数据，并将数据设置为其所管理视图组件的 dataProvider 属性。

再考虑一个持有 LoginVO 的 LoginProxy。LoginVO 是一个值对象，也就是简单的数据承载类。它可以写成下面这样：

```
package com.me.myapp.model.vo
{
        // 将这个 AS3 值对象映射到下面的远程类
        [RemoteClass(alias="com.me.myapp.model.vo.LoginVO")]

        [Bindable]
        public class LoginVO
        {
            public var username: String;
            public var password: String;
            public var authToken: String; // 凭据有效时，由服务器设置
        }
}
```

LoginProxy 公开了设置凭据、登录、注销，以及获取授权令牌的方法。在这种特定的身份验证方案中，登录之后发起的服务调用都需要包含这个令牌。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Proxy（代理）

与远程代理交互

LoginProxy:

```
package com.me.myapp.model
{
	import mx.rpc.events.FaultEvent;
	import mx.rpc.events.ResultEvent;
	import mx.rpc.remoting.RemoteObject;
	import org.puremvc.as3.interfaces.*;
	import org.puremvc.as3.patterns.proxy.Proxy;
	import com.me.myapp.model.vo.LoginVO;

	// 用于执行用户登录的代理
	public class LoginProxy extends Proxy implements IProxy {
		public static const NAME:String 		= 'LoginProxy';
		public static const LOGIN_SUCCESS:String 	= 'loginSuccess';
		public static const LOGIN_FAILED:String 	= 'loginFailed';
		public static const LOGGED_OUT:String 	= 'loggedOut';
		private var loginService: RemoteObject;

		public function LoginProxy () {
			super( NAME, new LoginVO ( ) );
			loginService = new RemoteObject();
			loginService.source = "LoginService";
			loginService.destination = "GenericDestination";
			loginService.addEventListener( FaultEvent.FAULT, onFault );
			loginService.login.addEventListener( ResultEvent.RESULT, onResult );
		}

		// 使用隐式 getter 转换数据对象的类型
		public function get loginVO( ) : LoginVO {
			return data as LoginVO;
		}

		// 登录值对象包含授权令牌时，用户即处于已登录状态
		public function get loggedIn():Boolean {
			return ( authToken != null );
		}

		// 登录后对服务的后续调用必须包含授权令牌
		public function get authToken():String {
			return loginVO.authToken;
		}
```

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Proxy（代理）

与远程代理交互

```
        // 设置用户凭据并登录，或先注销再重试
        public login( tryLogin:LoginVO ) : void {
                if ( ! loggedIn )    {
                        loginVO.username= tryLogin.username;
                        loginVO.password = tryLogin.password;
                } else {
                        logout();
                        login( tryLogin );
                }
        }

        // 注销时，只需清空 LoginVO
        public function logout( ) : void
        {
                if ( loggedIn ) loginVO = new LoginVO( );
                sendNotification( LOGGED_OUT );
        }

        // 向系统通知登录成功
        private function onResult( event:ResultEvent ) : void
        {
                setData( event.result ); // 可立即通过 loginVO 访问
                sendNotification( LOGIN_SUCCESS, authToken );
        }

        // 向系统通知登录故障
        private function onFault( event:FaultEvent) : void
        {
                sendNotification( LOGIN_FAILED,  event.fault.faultString );
        }
    }
}
```

LoginCommand 可以获取 LoginProxy，设置凭据，并调用 login 方法，从而调用服务。GetPrefsCommand 可以响应 LOGIN_SUCCESS 通知，从通知的 body 获取 authToken，然后调用下一个服务以获取用户偏好设置。

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。

Proxy（代理）

与远程代理交互

LoginCommand:

```
package com.me.myapp.controller {
	import org.puremvc.as3.interfaces.*;
	import org.puremvc.as3.patterns.command.*;
	import com.me.myapp.model.LoginProxy;
	import com.me.myapp.model.vo.LoginVO;

	public class LoginCommand extends SimpleCommand {
		override public function execute( note: INotification ) : void  {
			var loginVO : LoginVO = note.getBody() as LoginVO;
			var loginProxy: LoginProxy;
			loginProxy = facade.retrieveProxy( LoginProxy.NAME ) as LoginProxy;
			loginProxy.login( loginVO );
		}
	}
}
```

GetPrefsCommand:

```
package com.me.myapp.controller {
	import org.puremvc.as3.interfaces.*;
	import org.puremvc.as3.patterns.command.*;
	import com.me.myapp.model.LoginProxy;
	import com.me.myapp.model.vo.LoginVO;

	public class GetPrefsCommand extends SimpleCommand {
		override public function execute( note: INotification ) : void  {
			var authToken : String = note.getBody() as String;
			var prefsProxy : PrefsProxy;
			prefsProxy = facade.retrieveProxy( PrefsProxy.NAME ) as PrefsProxy;
			prefsProxy.getPrefs( authToken );
		}
	}
}
```

PureMVC 是由 Futurescale, Inc. 创建和维护的自由开源框架。版权所有 © 2006–08，保留部分权利。再使用须遵守 Creative Commons 3.0 署名美国许可协议。PureMVC、本文件，以及从 Futurescale 网站下载的任何培训资料或演示源代码均按“原样”提供，不附带任何明示或默示担保，包括但不限于适用于特定用途或不侵权的默示担保。